Textures & Images

DPG uses the Graphics Processing Unit (GPU) to create the graphical user interface(GUI) you see. To display an image, you must first create a texture with the image data that can then be uploaded to the GPU. These textures belong to a texture registry.

We offer 3 types of textures

- Static
- Dynamic
- Raw

This textures are then used in the following App Items

- **mvDrawImage**
- **mvImage**
- **mvImageButton**
- **mvImageSeries**

They are always 1D lists or arrays.

Using the keyword Show on the texture registry will open the texture registry.

Static Textures

Static textures are used for images that do not change often. They are typically loaded at startup. If they need to be updated, you would delete and recreate them. These accept python lists, tuples, numpy arrays, and any type that supports python's buffer protocol with contiguous data. Below is a simple example

```python
import dearpygui.dearpygui as dpg

dpg.create_context()

texture_data = []
for i in range(0, 100 * 100):
    texture_data.append(255 / 255)
    texture_data.append(0)
    texture_data.append(255 / 255)
    texture_data.append(255 / 255)

with dpg.texture_registry(show=True):
    dpg.add_static_texture(width=100, height=100, default_value=texture_data, tag="texture_tag")

with dpg.window(label="Tutorial"):
    dpg.add_image("texture_tag")


dpg.create_viewport(title='Custom Title', width=800, height=600)
dpg.setup_dearpygui()
dpg.show_viewport()
dpg.start_dearpygui()
dpg.destroy_context()
```

The texture can be deleted with *dpg.delete_item("texture_tag")*. However, for the tag/alias to be released items that use *"texture_tag"* (such as a plot series) must also be deleted.

Dynamic Textures

Dynamic textures are used for small to medium sized textures that can change per frame. These can be updated with `set_value` but the width and height must be the same as when the texture was first created. These are similar to raw textures except these perform safety checks and conversion. Below is a simple example

```python
import dearpygui.dearpygui as dpg

dpg.create_context()

texture_data = []
for i in range(0, 100 * 100):
    texture_data.append(255 / 255)
    texture_data.append(0)
    texture_data.append(255 / 255)
    texture_data.append(255 / 255)

with dpg.texture_registry(show=True):
    dpg.add_dynamic_texture(width=100, height=100, default_value=texture_data, tag="texture_tag")


def _update_dynamic_textures(sender, app_data, user_data):
    new_color = dpg.get_value(sender)
    new_color[0] = new_color[0] / 255
    new_color[1] = new_color[1] / 255
    new_color[2] = new_color[2] / 255
    new_color[3] = new_color[3] / 255

    new_texture_data = []
    for i in range(0, 100 * 100):
        new_texture_data.append(new_color[0])
        new_texture_data.append(new_color[1])
        new_texture_data.append(new_color[2])
        new_texture_data.append(new_color[3])

    dpg.set_value("texture_tag", new_texture_data)


with dpg.window(label="Tutorial"):
    dpg.add_image("texture_tag")
    dpg.add_color_picker((255, 0, 255, 255), label="Texture",
                         no_side_preview=True, alpha_bar=True, width=200,
                         callback=_update_dynamic_textures)


dpg.create_viewport(title='Custom Title', width=800, height=600)
dpg.setup_dearpygui()
dpg.show_viewport()
dpg.start_dearpygui()
dpg.destroy_context()
```

Raw Textures

Raw textures are used in the same way as dynamic textures. The main differences

- Only accepts arrays (numpy, python, etc.)
- No safety checks are performed.

latest

These textures are used for high performance applications that require updating large textures every frame. Below is a simple example

```
import dearpygui.dearpygui as dpg
import array

dpg.create_context()


texture_data = []
for i in range(0, 100 * 100):
    texture_data.append(255 / 255)
    texture_data.append(0)
    texture_data.append(255 / 255)
    texture_data.append(255 / 255)

raw_data = array.array('f', texture_data)

with dpg.texture_registry(show=True):
    dpg.add_raw_texture(width=100, height=100, default_value=raw_data,
format=dpg.mvFormat_Float_rgba, tag="texture_tag")


def update_dynamic_texture(sender, app_data, user_data):
    new_color = dpg.get_value(sender)
    new_color[0] = new_color[0] / 255
    new_color[1] = new_color[1] / 255
    new_color[2] = new_color[2] / 255
    new_color[3] = new_color[3] / 255

    for i in range(0, 100 * 100 * 4):
        raw_data[i] = new_color[i % 4]


with dpg.window(label="Tutorial"):
    dpg.add_image("texture_tag")
    dpg.add_color_picker((255, 0, 255, 255), label="Texture",
                         no_side_preview=True, alpha_bar=True, width=200,
                         callback=update_dynamic_texture)


dpg.create_viewport(title='Custom Title', width=800, height=600)
dpg.setup_dearpygui()
dpg.show_viewport()
dpg.start_dearpygui()
dpg.destroy_context()
```

Formats

The following formats are currently supported

Format	Static Texture	Dynamic Texture	Raw Texture
mvFormat_Float_rgba	✅	✅	✅
mvFormat_Float_rgb	•	•	✅*
mvFormat_Int_rgba	•	•	•
mvFormat_Int_rgb	•	•	•

Note

mvFormat_Float_rgb not currently supported on MacOS
More formats will be added in the future.

Loading Images

DPG provides the function `load_image` for loading image data from a file.

This function returns a tuple where

- 0 -> width
- 1 -> height
- 2 -> channels
- 3 -> data (1D array, mvBuffer)

On failure, returns **None**.

The accepted file types include:

- JPEG (no 12-bit-per-channel JPEG OR JPEG with arithmetic coding)
- PNG
- BMP
- PSD
- GIF
- HDR
- PIC
- PPM
- PGM

A simple example can be found below

```
import dearpygui.dearpygui as dpg

dpg.create_context()

width, height, channels, data = dpg.load_image("Somefile.png")

with dpg.texture_registry(show=True):
    dpg.add_static_texture(width=width, height=height, default_value=data, tag="texture_tag")

with dpg.window(label="Tutorial"):
    dpg.add_image("texture_tag")


dpg.create_viewport(title='Custom Title', width=800, height=600)
dpg.setup_dearpygui()
dpg.show_viewport()
dpg.start_dearpygui()
dpg.destroy_context()
```

Saving Images

New in 1.4. DPG provides the function `save_image` for saving image data to a file.

The image is a rectangle of pixels stored from left-to-right, top-to-bottom. Each pixel contains up to 4 components of data interleaved with 8-bits per channel, in the following order: 1=Y, 2=YA, 3=RGB, 4=RGBA. (Y is monochrome color.)

PNG creates output files with the same number of components as the input. The BMP format expands Y to RGB in the file format and does not output alpha.

Additional options will will be released with v1.4.1.

The accepted file types include:

- PNG
- JPG (new in v1.4.1)
- BMP (new in v1.4.1)
- TGA (new in v1.4.1)
- HDR (new in v1.4.1)

File type is determined by extension. Must be lowercase (png, jpg, bmp, tga, hdr

latest

A simple example can be found below

```python
import dearpygui.dearpygui as dpg

dpg.create_context()
dpg.create_viewport()
dpg.setup_dearpygui()

width, height = 255, 255

data = []
for i in range(width*height):
    data.append(255)
    data.append(255)
    data.append(0)

with dpg.window(label="Tutorial"):
    dpg.add_button(label="Save Image", callback=lambda:dpg.save_image(file="newImage.png",
width=width, height=height, data=data, components=3))

dpg.show_viewport()
while dpg.is_dearpygui_running():
    dpg.render_dearpygui_frame()

dpg.destroy_context()
```